

MeGPT

DIG 345 Final

RADICAL SOFTWARE · SPRING 2026 · DAVID YODER

yoder.ing/megpt

What is MeGPT?

MeGPT is a chat site styled like ChatGPT, but every message is routed to me through Discord and answered by hand. There is no language model behind it. The interface looks like an AI product — sidebar, message bubbles, typing indicator, persistent history — but the responses come from a person reading and replying on a phone.

It is now live at yoder.ing/megpt with guest access, Google sign-in, image uploads, and a Discord operator pipeline. Anyone can use it.

What Changed Since the Original Proposal

PIVOT 1 · CONCEPT

Deceptive AI → disclosed human chat

Original framing: users believe they're talking to ChatGPT; the "reveal" is part of the experience. Shipped framing: the about page and footer state plainly that MeGPT is a human. The visual mimicry is still the point — but the deception is not.

Why: deception-without-consent across a public URL is a different ethical posture than a Wizard-of-Oz study with briefed participants. Disclosure keeps the conceptual question (what does an AI interface feel like when there's no AI?) without the consent problem.

PIVOT 2 · MESSAGE CHANNEL

Telegram → Discord

The original webhook plan used Telegram. Shipped version uses a Discord bot (discord.js) that creates one thread per conversation in a parent channel. Conversation-to-thread mapping lives in Postgres, so when I reply in a thread the app knows which user's chat to route the message back to.

PIVOT 3 · HOSTING

Vercel + Supabase → Railway + Postgres

I had Vercel + Supabase in the proposal mostly out of habit. Once I dropped SSE for polling and reduced the data model to four tables, Supabase was overkill — Postgres directly through the pg driver is enough. Railway runs the Next.js app and the Postgres service in one project, with internal networking, which is simpler than the original split.

PIVOT 4 · SCOPE

Added: guest mode, image uploads

Guest mode wasn't in the original deck — I'd assumed Google OAuth as a gate. Lowering that wall meant more people would actually try it. Image uploads were scope creep, but useful: people send screenshots and I respond to them.

Audience

Classmates & Friends

The first wave. Most know it's me on the other end going in, which turns the site into a slow, weird messaging app rather than a Turing test.

Curious Visitors

People who land on the site through my portfolio. The about page tells them what's going on; the chat itself doesn't. Some still half-expect an AI.

Me, as Operator

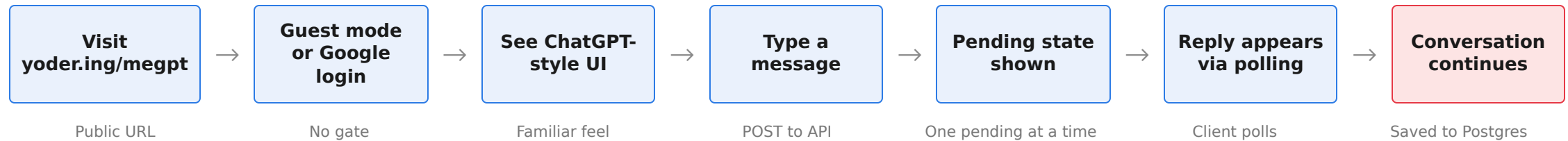
An audience I didn't think about in the proposal. Sitting on the other side of the interface for hours changed what the project is actually about.

Revised User Story

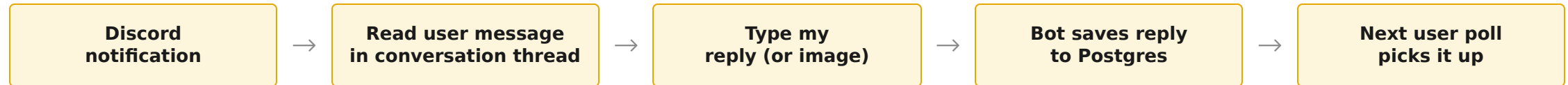
As a visitor to yoder.ing/megpt · **I want to** send a message in a familiar AI chat interface · **and get a reply** that arrives in 5-60 seconds, written by a person who knows they're being read as an AI. The friction between the interface (instant, infinite, confident) and the reality (a human, sometimes slow, sometimes asleep) is the work.

User Journey — As Shipped

USER VIEW



OPERATOR VIEW (ME, ON DISCORD)



What I learned about latency. The original deck guessed 2-6 seconds and planned a typing-indicator system to make timing feel right. In practice, the polling interval plus my real reading-and-typing time produces variable latency that's more convincing than any deliberate scheme. The give-away isn't speed — it's *shape*. Humans hedge, ask follow-ups, sound uncertain. AIs don't.

Data Architecture

FRONTEND

Next.js 16 + React 19

Chat UI with Tailwind 4
Google login (NextAuth, JWT) + guest mode
Polls for new messages on each conversation
Image upload component
lucide-react, motion for UI

SERVER

Next.js API + Discord Bot

POST handlers — save message, mirror to
Discord thread
Discord bot (discord.js) saves operator replies
to Postgres
One pending request per conversation
enforced
Image processing via sharp
Image serving at /api/images/:id

EXTERNAL / INFRA

Railway · Postgres · Discord

Postgres via pg (no ORM) — conversations,
messages, discord_threads, uploaded_images
Google OAuth
Discord parent channel + per-conversation
threads
Bun as runtime / package manager

Notable: no Supabase, no Vercel, no SSE. Supabase and Vercel were both in the original proposal and dropped — Postgres directly via pg is enough, and Railway hosts everything. SSE was the planned real-time mechanism; the shipped version polls instead. Simpler, fewer moving parts, and acceptable because nobody expects an "AI" to be instant anyway.

Technology Stack — As Shipped

CORE	Next.js 16 + React 19	Next.js 14 → Next.js 16, React 19, Tailwind 4. App Router + API routes.
RUNTIME	Bun	Package manager and runtime. Not in original deck. Faster installs and a single tool for build/dev/lint.
AUTH	NextAuth.js + Guest Mode	Google OAuth, JWT sessions. Added: guest mode with TTL'd temporary session IDs.
DB	PostgreSQL via pg	Supabase → Postgres directly through the pg driver. No ORM. Tables: conversations, messages, discord_threads, uploaded_images.
COMMS	Discord Bot (discord.js)	Telegram → Discord. One thread per conversation, mapping stored in Postgres.
UPDATES	Polling	SSE / EventSource → Frontend polls the messages endpoint. Simpler. The latency it adds is invisible — users already expect "AI" thinking time.
IMAGES	sharp + Postgres-stored bytes	User uploads and Discord image replies processed via sharp, served back through /api/images/:id.
DEPLOY	Railway	Vercel → Railway. Same project hosts the Next.js app and the Postgres service.

Data Sources & Usage

User-Generated

Source Chat messages, optional images
Stored Postgres (Railway)

Linked to Google account when signed in, or to a guest session ID otherwise. History persists. Privacy page acknowledges possible PII and that I can read all of it.

Auth Data

Source Google OAuth — email, name, avatar
Stored NextAuth session + users table

No passwords. Guest mode skips this entirely and creates only a session ID.

Operational

Source Timestamps, Discord thread IDs
Stored messages + conversations tables

Used to debug routing, look at latency in retrospect, and pair conversations with their Discord thread for moderation if needed.

What changed from the proposal: the about, privacy, and terms pages now exist and are linked from the homepage. Disclosure on the landing screen states plainly that "MeGPT, a human" is on the other end. That moves the project out of a Wizard-of-Oz deception frame and into something closer to a novelty chat service that happens to wear an AI's clothes.

Reflection

The interface is the lie, not the latency

I planned the entire deception around timing — typing indicators, 2-6 second delays, "natural latency simulation." None of that mattered. What gives away an AI is shape and certainty, not speed. ChatGPT answers everything; I have to choose to. That choice is visible in every reply.

Disclosure didn't ruin it

I worried that telling users it was a human up front would kill the project. It didn't. People still talk to it like an AI — they ask it to summarize things, debug code, recommend movies. The interface trains the behavior independently of what they "know." That's the more interesting finding than anything the deceptive version would have produced.

Operator burnout is a real design constraint

I baked it into the schema: only one pending user message per conversation at a time. That's a hard cap that prevents users (and me) from drowning. The system has a built-in human bottleneck no UI polish removes — which is, in retrospect, the most ChatGPT-comparing thing about the whole project.

What I'd do next

An "operator queue" view would help — right now I work straight off the Discord channel sidebar. A way to mark myself offline so users get a clear "the human is asleep" state instead of an indefinite pending. Reducing the polling interval, or moving to push-on-write once the rest is stable. A multi-operator mode is the obvious extension — but that defeats the project's point.